

— 모의해킹 결과 보고서 —

OWASP Juice Shop

웹 애플리케이션 보안 취약점 분석



작성자	남재근	과목	기업정보보호관리
제출 기한	~ 2026.07.05		

목 차

1. 모의해킹 수행 정보	3
(1) 개요	3
(2) 대상	4
(3) 수행 기간	4
2. 모의해킹 결과	4
(1) 총평	4
(2) [Login Admin] - SQL 인젝션 (SQL Injection)	5
(2-1) 취약점 개요	5
(2-2) 공격 과정 (proof of Concept)	5
(2-3) 공격 원리 분석	7
(2-4) 취약점 분석 및 대응 방안	7
(2-5) 보안 설정 및 소스코드 수정 방안 (대응 방안)	8
(3) [Zero Stars] - 부적절한 입력값 검증 및 파라미터 변조	10
(3-1) 취약점 개요	10
(3-2) 취약점 상세 설명	10
(3-3) 공격 시나리오 및 증적 (PoC)	11
(3-4) 보안 대책 및 대응 방안 (Remediation)	13
(4) [Admin Registration] 관리자 권한 등록 취약점	14
(4-1) 취약점 개요	14
(4-2) 공격 과정 (Proof of Concept)	15
(4-3) 공격 원리 분석	18
(4-4) 취약점 발생 원인	18
(4-5) 보안 대책 및 대응 방안	19

1. 모의해킹 수행 정보

(1) 개요

본 보고서는 웹 애플리케이션 환경에서 발생할 수 있는 주요 보안 취약점을 직접 식별하고, 이에 대한 기술적 방어 대책을 수립하는 역량을 기르기 위해 작성되었다. 실습을 위해 OWASP에서 제공하는 모의 해킹 플랫폼인 Juice Shop 인프라를 구축하고 수행한 모의해킹 및 취약점 분석 결과를 기술한다.

시스템 내에 숨겨진 점수판(Score Board) 페이지를 찾아내어 활성화하는 것을 시작으로 실습을 진행하였다. 최근 웹 서비스는 기능이 복잡해짐에 따라 개발자의 소스코드 상의 실수나 웹 서버의 설정 미흡으로 인해 심각한 보안 위협에 노출되는 경우가 많다. 따라서 이론으로만 접하던 웹 취약점들을 모의 훈련장에서 직접 테스트해보며 공격이 성공하는 구조적인 메커니즘을 상세히 분석하고자 한다.

본 실습의 최종 목적은 단순히 취약점을 발견하는 것에 그치지 않고, 이를 원천적으로 차단할 수 있는 시큐어 코딩(Secure Coding) 방안을 배우고 서버 설정을 올바르게 변경하는 등 방어자 관점에서의 실제적인 대응 능력을 체득하는 데 있다.

(2) 대상

진단 대상 항목	http://192.168.200.107:3000/#/
Platform	Web – OWASP Juice shop

(3) 수행 기간

수행 기간	2026년 06월 26일 ~ 2026년 07월 05일
-------	-------------------------------

2. 모의해킹 결과

(1) 총평

본 모의해킹 실습에서는 OWASP Juice Shop을 대상으로 다양한 웹 애플리케이션 취약점을 분석하고 공격을 수행하였다. Burp Suite와 브라우저 개발자 도구를 활용하여 HTTP 요청 및 응답을 분석하고, URL 및 파라미터 변조, REST API 접근, JSON 데이터 수정 등을 통해 여러 취약점을 확인하였다.

실습 과정에서는 SQL Injection, DOM XSS, Broken Access Control, Improper Input Validation, Sensitive Data Exposure 등 OWASP Top 10에 해당하는 주요 취약점을 직접 재현하였다. 이를 통해 클라이언트 측 검증만으로는 보안을 보장할 수 없으며, 서버 측 입력값 검증과 접근 권한 통제가 반드시 필요하다는 점을 확인하였다.

이번 실습을 통해 웹 애플리케이션 보안은 인증, 인가, 입력값 검증, 민감 정보 보호 및 시큐어 코딩이 함께 적용되어야 안전한 서비스를 구축할 수 있다는 점을 이해할 수 있었으며, 실제 취약점 분석 및 대응 방안을 학습하는 데 의미 있는 경험이 되었다.

(2) 취약점 1: [Login Admin] - SQL 인젝션 (SQL Injection)

진단 대상 항목	http://192.168.200.107:3000/#/login (고객 로그인)
취약점 명칭	SQL 인젝션 (SQL Injection) - 사용자 인증 우회 취약점

(2-1) 취약점 개요

개요: OWASP Juice Shop의 로그인 인증 모듈에서 사용자 입력값에 대한 검증 및 필터링 정책이 부재하여 발생하는 심각한 취약점이다.

공격자가 로그인 폼의 이메일 입력창에 SQL 명령어 구문으로 해석될 수 있는 특수문자(싱글 쿼테이션 ')와 SQL 주석 처리자(--))를 삽입하면, 웹 애플리케이션 백엔드가 이를 단순 데이터가 아닌 데이터베이스 쿼리 명령어로 오인하여 실행한다.

이를 통해 공격자는 비밀번호를 전혀 모르는 상태에서도 데이터베이스 내 최상위 권한을 가진 관리자(Admin) 계정으로 인증을 우회하여 세션을 탈취할 수 있다.

(2-2) 공격 과정 (Proof of Concept)

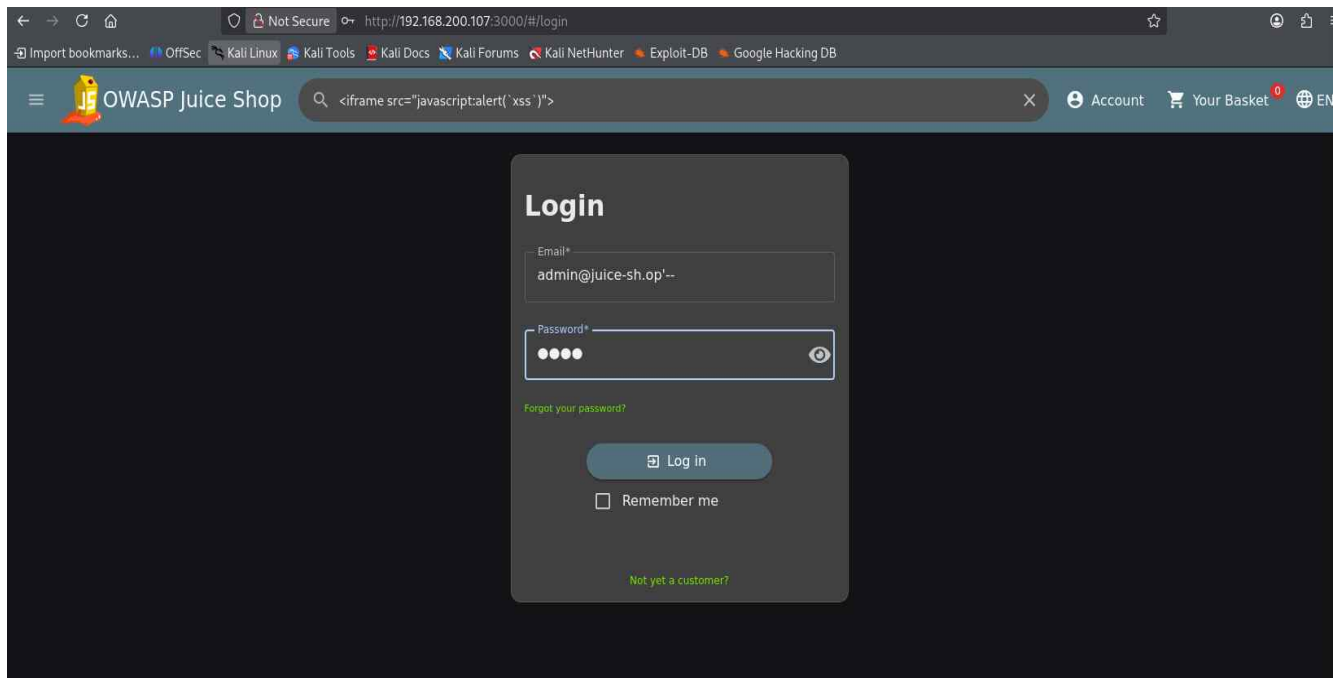
대상 로그인 페이지 접근: 주스숍 메인 화면 우측 상단의 Account 메뉴를 클릭한 후 Login 화면(http://192.168.200.107:3000/#/login)으로 이동한다.

공격 페이로드 주입: * Email 입력창: admin@juice-sh.op'--

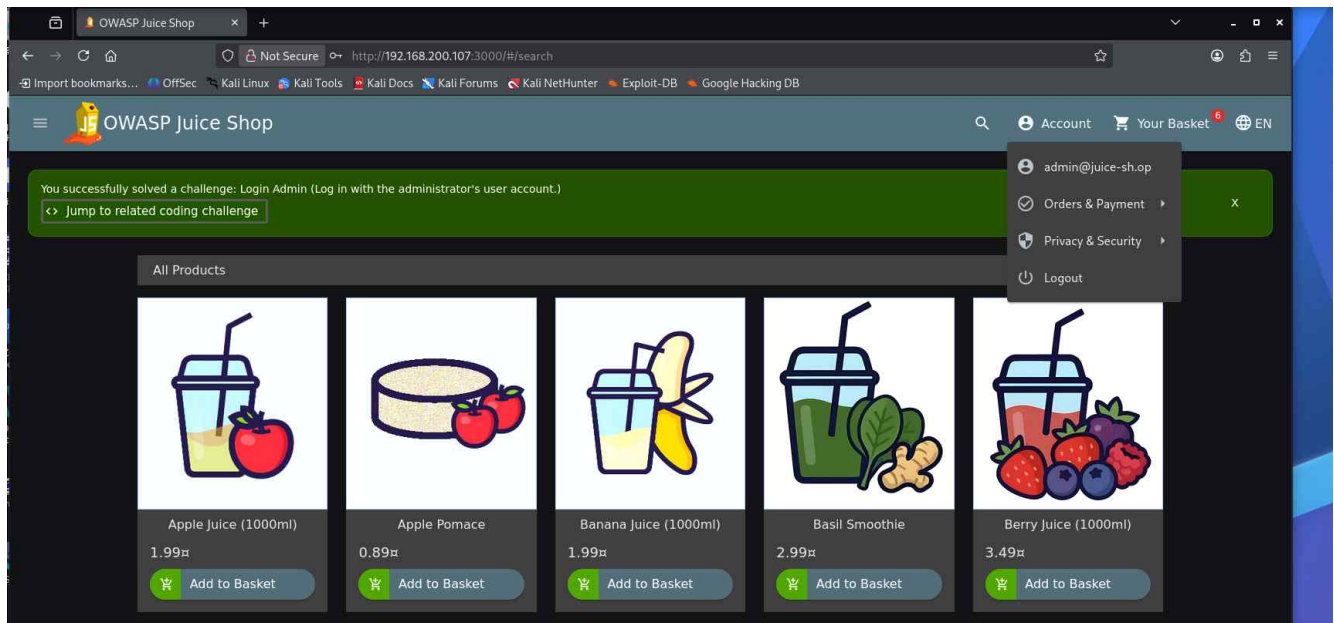
Password 입력창: 임의의 패스워드 입력 (예: 1234)

인증 우회 실행: 입력 후 Log in 버튼을 클릭한다.

[로그인 전]



[로그인 후]



이렇게 로그인 성공한 것을 볼 수 있다.

(2-3) 공격 원리 분석

OWASP Juice Shop 백엔드(Node.js/Sequelize ORM 환경) 내부에서는 사용자가 입력한 데이터베이스 조회를 위해 다음과 같은 SQL 구조를 동적으로 생성한다.

```
SELECT * FROM Users WHERE email = 'admin@juice-sh.op'--' AND password =  
'7110eda4d09e062aa5e4a390b0a572ac0d2c0220'
```

이때 공격자가 주입한 admin@juice-sh.op'--의 '문자에 의해 email 값을 감싸던 기존 SQL의 쿼테이션이 강제로 닫히게 된다.

이어서 등장하는 두 개의 하이픈(--)은 SQLite/SQL 환경에서 '이후의 모든 구문을 주석(Ignore) 처리하라'는 명령어로 동작한다.

결과적으로 뒤쪽에 위치한 패스워드 검증 로직인 AND password = '...' 구문이 통째로 증발하여 무력화되며, 데이터베이스는 WHERE email = 'admin@juice-sh.op' 조건만 참(True)으로 판단하여 비밀번호 일치 여부와 상관없이 최고 관리자의 회원 정보를 반환하고 인증을 승인하게 된다.

(2-4) 취약점 분석 및 대응 방안

△ 취약점 발생 원인 (소스코드 구조 분석)

해당 취약점이 발생하는 근본적인 원인은 백엔드 소스코드 내에서 데이터베이스와 상호작용할 때, 사용자가 전달한 파라미터를 인자화하지 않고 문자열을 그대로 결합하는 동적 쿼리(String Concatenation) 방식을 사용하기 때문이다.

취약한 소스코드 구조 예시:

```
// 사용자가 입력한 req.body.email 값을 문자열 템플릿(`${}`)을 통해 쿼리에 직접 결합  
  
models.sequelize.query(  
  
  `SELECT * FROM Users WHERE email = '${req.body.email}' AND password =  
  '${security.hash(req.body.password)}'`,  
  
  { type: models.sequelize.QueryTypes.SELECT }  
  
)
```

위와 같이 구현될 경우 데이터베이스 엔진은 컴파일 단계에서 명령과 데이터를 구분하지 못하므로 입력값에 포함된 특수문자가 쿼리의 구조를 변조할 수 있게 된다.

(2-5) 보안 설정 및 소스코드 수정 방안 (대응 방안)

① 매개변수화된 쿼리 (Parameterized Query / Prepared Statement) 적용

SQL 인젝션을 원천 차단하는 가장 확실한 표준 보안 대책은 쿼리의 '구문 구조'를 미리 컴파일해 두고, 사용자 입력값은 단순 '리터럴 데이터(텍스트)'로만 바인딩하는 매개변수화 기법을 적용하는 것이다.

안전한 시큐어 코딩 가이드 (Sequelize ORM 내장 메서드 활용):

// 안전한 구현: 문자열 결합을 배제하고 ORM의 선언적 구조체(where)를 사용

```
models.User.findOne({  
  
  where: {  
  
    email: req.body.email,  
  
    password: security.hash(req.body.password)  
  
  }  
  
})
```

이렇게 수정하면 공격자가 이메일 창에 아무리 '나 --' 같은 특수문자를 주입하더라도 데이터베이스는 이를 명령어가 아닌 무해한 단순 문자열 데이터("admin@juice-sh.op'--") 자체로 인식하여 처리하므로 공격이 무력화된다.

(3) 취약점 2 : [Zero Stars] 취약점

(3-1) 취약점 개요

진단 대상 항목	OWASP Juice Shop - Customer Feedback (고객 피드백 평점 기능)
취약점 명칭	부적절한 입력값 검증 (Improper Input Validation) / 파라미터 변조 (Parameter Tampering)

(3-2) 취약점 상세 설명

본 취약점은 클라이언트 측(웹 브라우저 프론트엔드)에서 수행하는 입력값 검증 로직을 신뢰하고, 백엔드 서버 측에서 재검증을 수행하지 않을 때 발생하는 전형적인 보안 설정 미흡(Client-Side Validation Bypass) 취약점이다.

대상 애플리케이션의 피드백 평점 시스템은 UI 상에서 별점 컴포넌트를 사용해 1점부터 5점까지만 선택이 가능하도록 제한하고 있다. 하지만 서버로 전송되는 HTTP 데이터 패킷을 프록시 툴을 이용해 전송 과정에서 인위적으로 변경할 경우, 프론트엔드가 설정한 비즈니스 로직 경계값을 무력화하고 개발자가 의도하지 않은 비정상 데이터(0점)를 강제로 가로채 서버에 삽입할 수 있다.

(3-3) 공격 시나리오 및 증적 (PoC)

탐색 및 취약점 확인

- 사용자는 /#/contact (Customer Feedback) 메뉴로 이동하여 의견(Comment) 작성 및 CAPTCHA 인증 절차를 정상 수행한다.
- UI 컴포넌트 구조상 평점(Rating) 마우스 클릭으로 선택할 수 있는 최하점은 1점으로 바인딩되어 있으며, 전송 버튼(Submit)은 모든 적합성이 검증되어야 활성화된다.

프록시 도구(Burp Suite)를 통한 패킷 인터셉트 및 위조

- 로컬 프록시 도구인 Burp Suite의 인터셉트(Intercept) 기능을 활성화한 후, 브라우저에서 [Submit] 버튼을 클릭한다.
- 클라이언트 브라우저에서 웹 서버로 유입되는 실제 전송 패킷을 데이터 구조(JSON Body)를 식별한다. (버프 스위트로 결국 가로채는 것)

변조 데이터 주입 및 전송

인터셉트된 HTTP Request Body 영역의 파라미터 중 평점 점수를 나타내는 rating 인자 값을 기존 양의 정수에서 0으로 변조한다.

[원본 전송 데이터 (Original Request Body)]

The screenshot shows the Burp Suite interface with the 'Intercept' tab selected. A request is intercepted from http://192.168.200.107:3000/api/Feedbacks/. The request details are shown in the 'Request' tab, displaying the raw data in JSON format:

```
{
  "userId": 1,
  "captchaId": 13,
  "captcha": "63",
  "comment": "test (**in@juice-sh.op)",
  "rating": 1
}
```

The 'rating' field is highlighted with a red box, indicating its value is 1.

“rating” : 1

[변조 주입 데이터 (Modified Request Body)]

The screenshot shows the Burp Suite interface with the 'Intercept' tab selected. A request is intercepted from http://192.168.200.107:3000/api/Feedbacks/. The request details are shown in the 'Request' tab, displaying the raw data in JSON format:

```
{
  "userId": 1,
  "captchaId": 13,
  "captcha": "63",
  "comment": "test (**in@juice-sh.op)",
  "rating": 0
}
```

The 'rating' field is highlighted with a red box, indicating its value is 0.

“rating” : 0

공격 결과 확인

- 변조된 패킷을 웹 서버로 포워딩(Forward) 처리 시, 백엔드에서 에러 거부 없이 HTTP/1.1 201 Created 응답과 함께 가공되지 않은 별점 0점 피드백 데이터를 그대로 수용한다.
- 결과적으로 시스템 관리자 점수판 및 스코어보드에서 'Zero Stars' 챌린지가 성공적으로 Solved 상태로 변경됨을 확인하였다.

(3-4) 보안 대책 및 대응 방안 (Remediation)

심층 방어 기반의 서버 측 검증 (Server-Side Validation) 구현

보안의 대원칙인 '클라이언트로부터 인입되는 모든 입력값은 신뢰하지 않는다(Trust No One)'를 적용해야 한다. 자바스크립트 등 프론트엔드 검증은 오직 사용자 편의성(UX) 증대 목적으로만 제한하고, 실질적인 입력값의 범위 검증은 백엔드 소스코드 단에서 반드시 재검증 절차를 거치도록 수정해야 한다.

안전한 검증 로직 예시 (Node.js / Express 예시)

```
// Feedback API Controller 예시
router.post('/api/Feedbacks', (req, res) => {
  const { rating, comment } = req.body;

  // 백엔드 단에서 평점 도메인 범위(1 ~ 5) 검증 예외 처리 추가
  if (!rating || rating < 1 || rating > 5) {
    return res.status(400).json({
      status: 'error',
      message: '부적절한 평점 값입니다. 평점은 1점에서 5점 사이여야 합니다.'
    });
  }

  // 비즈니스 로직 및 DB 저장 수행...
});
```

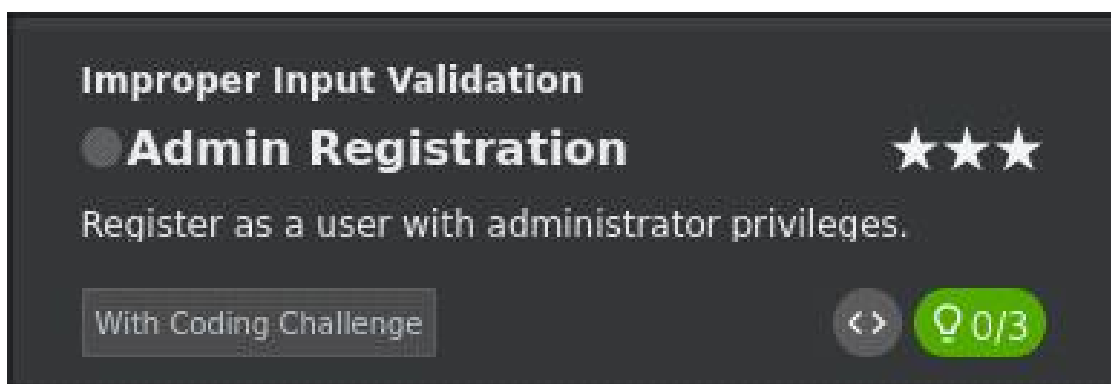
진단 대상 항목	OWASP Juice Shop - User Registration / POST /api/Users/
취약점 명칭	부적절한 입력값 검증(Improper Input Validation) / Mass Assignment(대량 할당

(4) 취약점 3 : [Admin Registration] 관리자 권한 등록 취약점

(4-1) 취약점 개요

본 취약점은 회원가입 API가 클라이언트에서 전달되는 JSON Body 값을 충분히 제한하지 않고 서버 내부 사용자 객체에 그대로 매핑하면서 발생하는 권한 상승 취약점이다. 일반 회원가입 화면에서는 관리자 권한을 선택할 수 있는 UI가 제공되지 않지만, Burp Suite를 이용해 회원가입 요청을 가로챈 뒤 JSON Body에 role 필드를 강제로 추가하면 일반 사용자가 관리자 권한을 가진 계정으로 등록될 수 있다.

즉, 사용자가 입력해서는 안 되는 내부 권한 속성(role)을 서버가 신뢰하고 처리했기 때문에 발생한 Mass Assignment 취약점으로 볼 수 있다.



[Admin Registration 챌린지 항목]

1. You have to assign the unassignable.
2. Register as an ordinary user to learn what API endpoints are involved in this use case.
3. Think of the simplest possible implementations of a distinction between regular users and administrators.

[Admin Registration 힌트: 할당할 수 없는 값을 할당하라는 내용]

(4-2) 공격 과정 (Proof of Concept)

- ① Juice Shop 회원가입 페이지로 이동하여 일반 사용자 계정 정보를 입력한다.
- ② Burp Suite의 Proxy → Intercept 기능을 ON으로 설정한다.
- ③ Register 버튼을 클릭하여 POST /api/Users/ 요청을 Burp Suite에서 가로챈다.
- ④ 요청 Body의 마지막 항목인 securityAnswer 뒤에 쉽표(.)를 추가한다.
- ⑤ 이어서 "role":"admin" 필드를 추가한 뒤 Forward를 눌러 서버로 전송한다.
- ⑥ 서버가 해당 값을 거부하지 않고 사용자 객체에 반영하면 관리자 권한 계정 등록이 성공하며 Admin Registration 챌린지가 해결된다.

User Registration

Email*
test2@test.com

Password*
•••••

! Password must be 5-40 characters long. 5/20

Repeat Password*
•••••

5/40

Show password advice

Security Question *
Your eldest siblings middle name?

! This cannot be changed later!

Answer*
12345

Register

[회원가입 페이지에서 일반 사용자 정보 입력]

Interception Forward Drop

Time Type Direction Method URL

05:35:31 306월 ... HTTP → Request POST http://192.168.200.107:3000/api/Users/

Request

Pretty Raw Hex

```

{
  "password": "12345",
  "passwordRepeat": "12345",
  "securityQuestion": {
    "id": 1,
    "question": "Your eldest siblings middle name?",
    "createdAt": "2026-06-30T08:25:09.198Z",
    "updatedAt": "2026-06-30T08:25:09.198Z"
  },
  "securityAnswer": "12345",
  "role": "admin"
}

```

16 }
17 }

Search

[Burp Suite에서 role 필드를 추가한 최종 요청]

원본 요청 데이터

```
{
  "email": "test2@test.com",
  "password": "12345",
  "passwordRepeat": "12345",
  "securityQuestion": {
    "id": 1,
    "question": "Your eldest siblings middle name?",
    "createdAt": "2026-06-30T08:25:09.198Z",
    "updatedAt": "2026-06-30T08:25:09.198Z"
  },
  "securityAnswer": "12345"
}
```

변조 요청 데이터

```
{
  "email": "test2@test.com",
  "password": "12345",
  "passwordRepeat": "12345",
  "securityQuestion": {
    "id": 1,
    "question": "Your eldest siblings middle name?",
    "createdAt": "2026-06-30T08:25:09.198Z",
    "updatedAt": "2026-06-30T08:25:09.198Z"
  },
  "securityAnswer": "12345",
  "role": "admin"
}
```

※ 주의: JSON 문법상 "securityAnswer": "12345" 뒤에는 반드시 쉼표(,)가 있어야 한다.

쉼표가 누락되면 서버는 권한 검증 단계까지 가지 못하고 JSON 파싱 오류(Expected , or } after property value)를 반환한다.

(4-3) 공격 원리 분석

회원가입 기능은 원래 email, password, passwordRepeat, securityQuestion, securityAnswer와 같은 일반 사용자 등록 정보만 처리해야 한다.

그러나 서버가 요청 Body 전체를 사용자 모델에 직접 바인딩하면 공격자는 UI에 존재하지 않는 내부 속성도 임의로 추가할 수 있다.

이번 실습에서는 요청 Body에 "role":"admin" 값을 추가하였다.

서버 측에서 허용 필드 목록을 기준으로 필터링하지 않았기 때문에 role 속성이 사용자 객체에 포함되었고, 이로 인해 일반 회원가입 과정에서 관리자 권한 계정이 생성되는 결과가 발생하였다.

(4-4) 취약점 발생 원인

- 클라이언트가 전달한 JSON 데이터를 서버가 과도하게 신뢰함
- 회원가입 API에서 허용 가능한 필드만 처리하는 화이트리스트 검증이 부족함
- 권한 정보(role)를 서버 내부에서만 관리하지 않고 외부 요청 값으로 덮어쓸 수 있음
- 프론트엔드 UI 제한을 보안 통제 수단으로 오해함

(4-5) 보안 대책 및 대응 방안

① 화이트리스트 기반 입력 필드 제한

회원가입 API에서는 email, password, securityQuestion, securityAnswer 등 실제로 필요한 필드만 명시적으로 추출하여 처리해야 한다. role, isAdmin, permissions와 같은 권한 관련 필드는 클라이언트 요청에서 수신하더라도 무시하거나 삭제해야 한다.

```
// 취약한 예시: 요청 Body 전체를 그대로 사용
const user = await User.create(req.body)
```

```
// 안전한 예시: 허용된 필드만 명시적으로 추출
const { email, password, securityQuestion, securityAnswer } =
  req.body
const user = await User.create({
  email,
  password: hash(password),
  securityQuestion,
  securityAnswer,
  role: 'customer' // 권한은 서버에서 고정
})
```

② 권한 값은 서버에서만 설정

일반 회원가입으로 생성되는 모든 계정은 기본적으로 customer 또는 user 권한으로 고정해야 하며, 관리자 권한은 별도의 관리자 전용 기능과 승인 절차를 통해서만 부여해야 한다.

③ 서버 측 권한 검증 강화

관리자 권한이 필요한 API는 JWT나 세션 정보에 포함된 서버 검증 완료 권한을 기준으로 접근을 제한해야 하며, 사용자가 요청 Body에 전달한 role 값은 신뢰하면 안 된다.

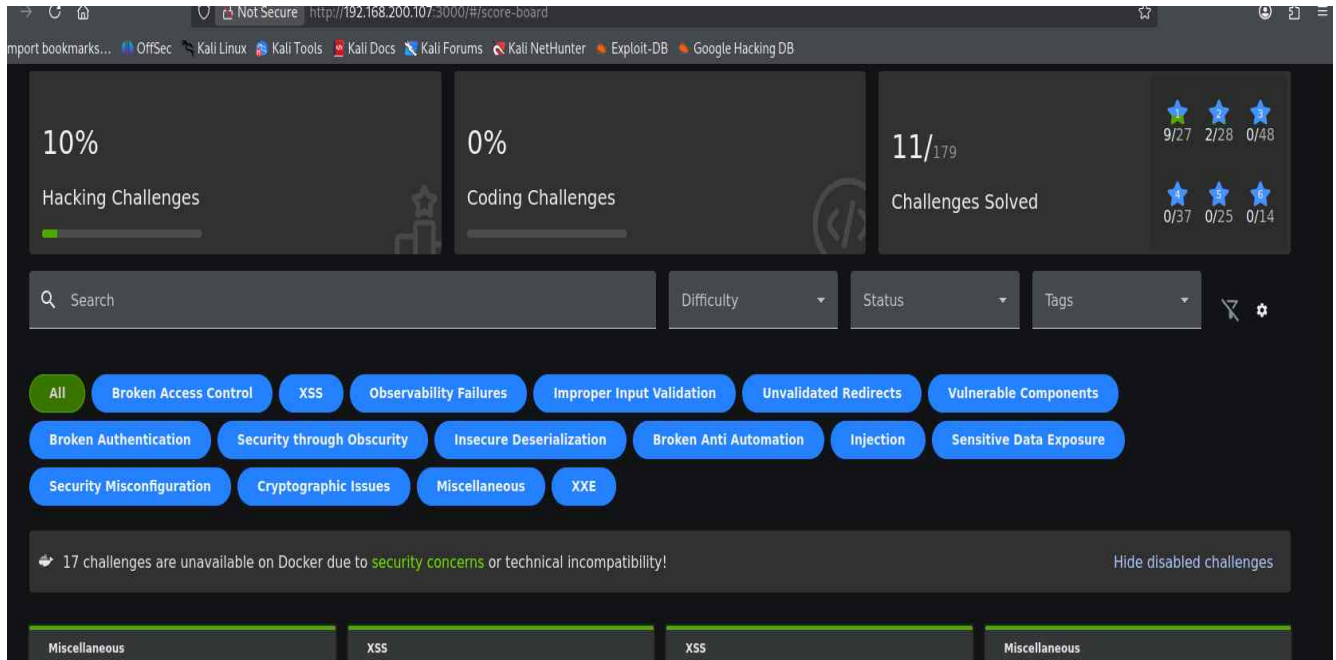
④ Mass Assignment 방지

ORM 또는 프레임워크 사용 시 모델 전체를 자동 바인딩하지 말고 DTO(Data Transfer Object), Validation Schema, allow-list 방식으로 입력 필드를 제한해야 한다.

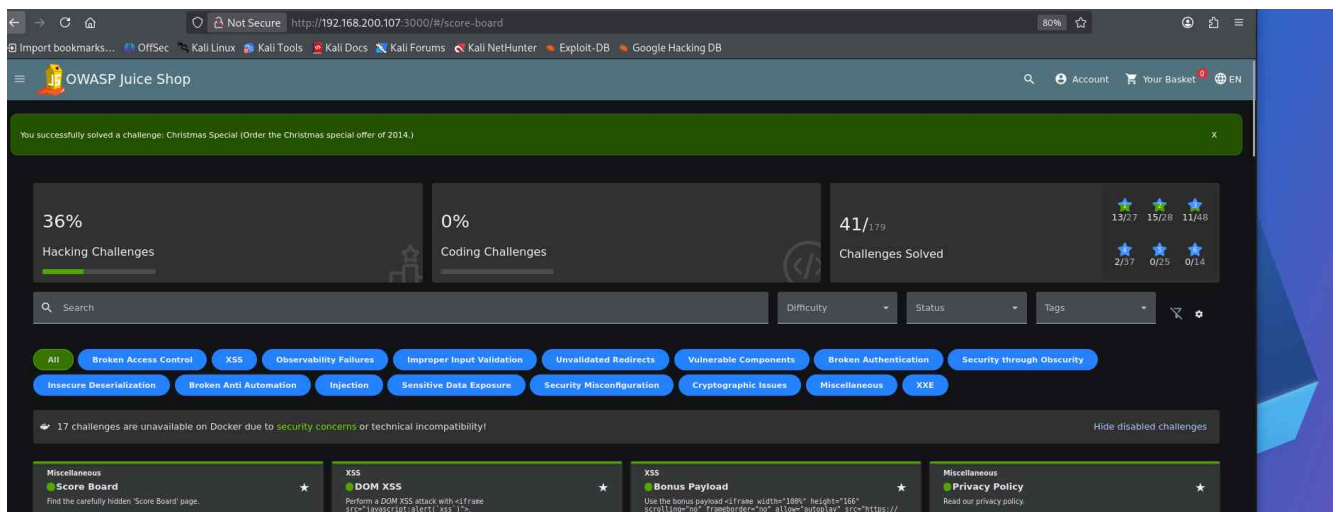
※ Admin Registration 취약점은 단순히 회원가입 기능의 문제가 아니라 서버가 사용자의 입력값을 어디까지 신뢰할 것인지에 대한 접근 통제 문제이다.

클라이언트 화면에 관리자 권한 선택지가 없더라도 HTTP 요청은 사용자가 자유롭게 조작할 수 있으므로, 서버는 반드시 허용된 입력값만 처리하고 권한 관련 속성은 서버 내부 로직에서만 관리해야 한다.

현재 주스샵 점수판 순서대로 진행 중.



주스샵 기록이 2번 날라갔으나.. 2번째에 기록이 날라간 건 복구했습니다.. 시간이 많이 걸리네요 그래서 이정도까지만 하고 마무리했습니다 ㅠ



그래도 한가지 알게된 점은 자동으로 받는 ip 였는데 ip 주소가 계속 바뀌는 걸 확인하여 바뀔 때마다 기록이 날라가는 것을 확인했는데요. 그래서 혹시나 이전 주소를 알고 있다면 주소를 이전 주소의 ip 로 고정시켜 다시 서버를 재부팅하면 기록이 복구되지 않을까 했어서 멘토님께 여쭙봤는데 대성공이었습니다. 결국 그래도 복구하여 지금의 결과 정도까지라도 온게 아닌가 싶습니다....